FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de tratamiento de Datos

Diseño de Bases de Datos Relacionales

Año: 2025

FADENA
FACULTAD DE DEFENSA NACIONAL
Universidad de la Defensa Nacional

Visión general del proceso de diseño

La tarea de creación de aplicaciones de bases de datos es una labor compleja, que implica varias fases, como el diseño del esquema de la base de datos, el diseño de los programas que tienen acceso a los datos y los actualizan y el diseño del esquema de seguridad para controlar el acceso a los datos. Las necesidades de los usuarios desempeñan un papel central en el proceso de diseño. En este caso nos centraremos en el diseño del esquema de la base de datos.

Fases del diseño

El diseñador de la base de datos debe interactuar con los usuarios para comprender las necesidades de la aplicación, realizar una representación de alto nivel de esas necesidades que pueda ser comprendida por los usuarios y luego traducir esos requisitos a niveles inferiores del diseño. Los modelos de datos de alto nivel sirven a los diseñadores de bases de datos ofreciéndoles un marco conceptual en el que especificar de forma sistemática los requisitos de datos de los usuarios de la base de datos y una estructura para la base de datos que satisfaga estos requisitos.

- El primer paso es la **recopilación de requisitos y el análisis**. Durante este paso, los diseñadores de bases de datos entrevistan a los potenciales usuarios de la base de datos para comprender y documentar sus **requisitos en cuanto a datos**. Esto implica

interactuar intensamente con los expertos y los usuarios del dominio. El resultado de este paso es un conjunto por escrito de los requisitos del usuario. Estos requisitos se deben plasmar en un formulario lo más detallado y completo posible. En paralelo al estudio de estos requisitos, resulta útil especificar los requisitos funcionales de la aplicación, que consisten en las operaciones (o transacciones) definidas por el usuario que se aplicarán a la base de datos, incluyendo las recuperaciones y las actualizaciones. En el diseño de software, es frecuente utilizar los diagramas de flujo de datos, diagramas de secuencia, escenarios y otras técnicas para especificar los requisitos funcionales.

- El siguiente paso es crear un **esquema conceptual** para la base de datos, mediante un modelo de datos conceptual de alto nivel. Este paso se denomina **diseño conceptual**. El esquema conceptual es una **descripción concisa de los requisitos de datos** por parte de los usuarios e incluye descripciones detalladas de las entidades que se representan en la base de datos, sus atributos, las relaciones entre ellas y las restricciones que las afectan; se expresan utilizando los conceptos proporcionados por el modelo de datos de alto nivel, como puede ser el modelo entidad-relación. Como estos conceptos no incluyen detalles de implementación, normalmente son más fáciles de entender y se pueden utilizar para comunicarse con usuarios no técnicos. Generalmente, la fase de diseño conceptual **da lugar a la creación de un diagrama entidad-relación**

que ofrece una representación gráfica del esquema. El diseñador revisa el esquema para confirmar que realmente se satisfacen todos los requisitos y que no entran en conflicto entre sí. También puede examinar el diseño para eliminar características redundantes. Esta metodología permite a los diseñadores de bases de datos concentrarse en especificar las propiedades de los datos, sin tener que preocuparse por los detalles del almacenamiento físico.

- El siguiente paso es el paso desde el modelo abstracto de datos a la implementación real de la base de datos mediante un DBMS comercial. Se divide en de dos fases de diseño finales:
    - En la fase de **diseño lógico** el diseñador traduce el esquema conceptual de alto nivel al modelo de datos de la implementación del sistema de bases de datos que se va a usar. El modelo de implementación de los datos **suele ser el modelo relacional**, y este paso suele consistir en la **traducción (o mapeo)** del esquema conceptual definido mediante el modelo entidad-relación en un esquema de relación de base de datos en el modelo de datos de implementación del DBMS.
    - Finalmente, el diseñador usa el esquema de base de datos resultante propio del sistema en la siguiente fase de **diseño físico**, en la que se especifican las características físicas de

la base de datos. Entre estas características están la forma de organización de los archivos y las estructuras de almacenamiento interno, los índices, etc.

El esquema físico de la base de datos puede modificarse con relativa facilidad una vez creada la aplicación. Sin embargo, las modificaciones del esquema lógico suelen resultar más difíciles de llevar a cabo, ya que pueden afectar a varias consultas y actualizaciones dispersas por todo el código de la aplicación. Por tanto, es importante llevar a cabo con cuidado la fase de diseño de la base de datos antes de crear el resto de la aplicación de bases de datos.

El modelo entidad-relación

El modelo de datos entidad–relación (E-R) se desarrolló para facilitar el diseño de bases de datos permitiendo la especificación de un esquema que representa la estructura lógica global de la base de datos. El modelo de datos E-R es uno de los diferentes modelos de datos semánticos, y permite definir un modelo conceptual que representa un conjunto de conceptos de la realidad y como se asocian entre ellos. Muchas herramientas de diseño de bases de datos se basan en los conceptos del modelo E-R.

Los modelos conceptuales deben ser buenas herramientas para representar la realidad, por lo que deben poseer las siguientes cualidades (el modelo entidad-relación lo hace):

- **Expresividad:** deben tener suficientes conceptos para expresar perfectamente la realidad.
- **Simplicidad:** deben ser simples para que los esquemas sean fáciles de entender.
- **Minimalidad:** cada concepto debe tener un significado distinto.
- **Formalidad:** todos los conceptos deben tener una interpretación única, precisa y bien definida, con el uso estricto de la sintaxis del lenguaje del modelo.

El modelo de datos E-R emplea tres conceptos básicos: entidades, relaciones y atributos.

Entidades

El objeto básico representado por el modelo ER es una **entidad**, que es un concepto u objeto del mundo real con una existencia independiente, distinguible de todos los demás objetos. Una entidad puede ser un objeto con una existencia física -concreto- (por ejemplo, una persona en particular, un auto, un libro o un empleado) o puede ser un objeto con una existencia conceptual -abstracto- (por ejemplo, una empresa, un préstamo o un curso universitario). Cada entidad tiene **atributos** (conjunto de propiedades particulares que la describen). Por ejemplo, una entidad Empleado se puede describir mediante el número de legajo, el nombre, la edad, la dirección, el sueldo y el trabajo que desempeña.

Un **conjunto de entidades** es un conjunto de entidades del mismo tipo que comparten las mismas propiedades, o atributos. La designación de un

atributo para un conjunto de entidades expresa que la base de datos almacena información parecida relativa a cada entidad del conjunto de entidades; sin embargo, cada entidad puede tener su propio valor para cada atributo. Un **tipo de entidad** define una colección (o conjunto) de entidades que tienen los mismos atributos, describiendo el esquema de un conjunto de entidades que comparten la misma estructura.

Atributos

Para cada atributo hay un conjunto de valores permitidos, denominados **dominio** o **conjunto de valores** de ese atributo. El dominio del atributo nombre_cliente puede ser el conjunto de todas las cadenas de texto de una cierta longitud.

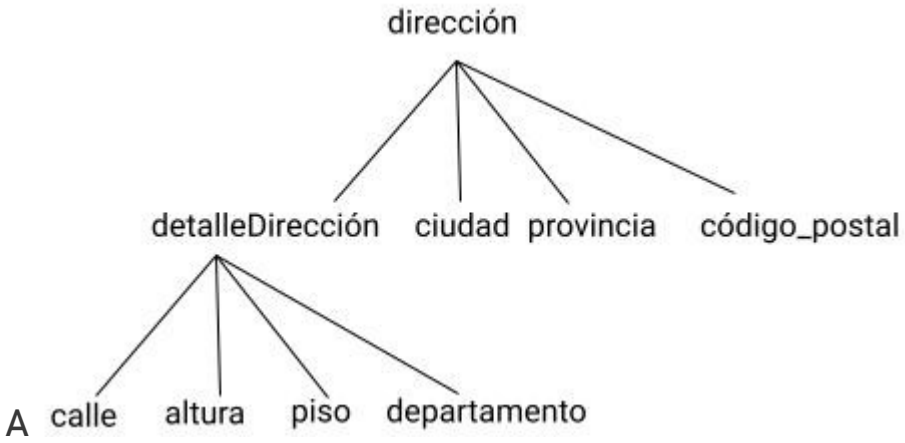
Dado que el conjunto de entidades puede tener varios atributos, cada entidad se puede describir mediante un conjunto de pares (atributo,valor), un par por cada atributo del conjunto de entidades.

Cada atributo, tal y como se usa en el modelo E-R, se puede caracterizar por los siguientes tipos.

- Atributos **simples** y **compuestos**. Los atributos que no son divisibles se denominan atributos simples o atómicos, representan los atributos más básicos con significados independientes. Los atributos compuestos, en cambio, se pueden dividir en subpartes más pequeñas (es decir, en otros atributos). Por ejemplo, el

atributo nombre puede estar estructurado como un atributo compuesto por nombre, primer_apellido y segundo _apellido.

Los atributos compuestos son útiles para modelar situaciones en las que un usuario se refiere a veces al atributo compuesto como una unidad (agrupando atributos relacionados, lo que hace que los modelos sean más claros), pero otras veces se refiere específicamente a sus componentes. Si se hace referencia al atributo compuesto como un todo, no hay necesidad de subdividirlo en atributos componentes. Por ejemplo, si no hay necesidad de referirse a los componentes individuales de una dirección (código postal, calle, etcétera), entonces la dirección entera se puede designar como un atributo simple. Los atributos compuestos pueden formar una jerarquía; por ejemplo, en el atributo compuesto dirección, el componente detalleDirección puede dividirse a su vez en calle, altura, piso y departamento.



- Atributos **monovalor** y **multivalor**. La mayoría de los atributos tienen un solo valor para una entidad en particular; dichos atributos

reciben el nombre de monovalor o de un solo valor. Por ejemplo, edad es un atributo monovalor de una persona. En algunos casos, un atributo puede tener un conjunto de valores para la misma entidad concreta. Por ejemplo, conjunto de entidades empleado con el atributo número_teléfono. Cada empleado puede tener cero, uno o varios números de teléfono, y empleados diferentes pueden tener diferente cantidad de teléfonos. Se dice que este tipo de atributo es **multivalor**. Un atributo multivalor puede tener límites superior e inferior para restringir el número de valores permitidos para cada entidad individual.

- Atributos **almacenados** y **derivados**. En algunos casos, dos (o más) valores de atributo están relacionados (por ejemplo, los atributos Edad y FechaNac de una persona). Para una entidad de persona en particular, el valor de Edad puede determinarse a partir de la fecha actual (el día de hoy) y el valor de FechaNac de esa persona. El atributo Edad se denomina entonces **atributo derivado** y se dice que se ha derivado del atributo FechaNac, que es el denominado **atributo almacenado**. Algunos valores de atributo se pueden derivar de entidades relacionadas; por ejemplo, un atributo NumEmpleados de una entidad departamento puede derivarse contando el número de empleados relacionados con (o que trabajan para) ese departamento.

- **Valores NULL** (**nulos**). Los atributos toman valores nulos cuando las entidades no tienen ningún valor para ese atributo. El valor nulo

puede indicar “no aplicable”, es decir, que el valor no existe para esa entidad. Por ejemplo, una persona puede no tener un segundo nombre de pila. Nulo puede también designar que el valor del atributo es desconocido. Un valor desconocido puede ser falta (el valor existe, pero no se tiene esa información) o desconocido (no se sabe si ese valor existe realmente o no). Por ejemplo, si el valor nombre de un cliente dado es nulo, se da por supuesto que el valor es falta, ya que todos los clientes deben tener nombre.

- **Atributos clave** de un tipo de entidad. Una restricción importante de las entidades de un tipo de entidad es la clave o restricción de unicidad de los atributos. Un tipo de entidad normalmente tiene un atributo cuyos valores son distintos para cada entidad individual del conjunto de entidades. Dicho atributo se denomina atributo **clave**, y sus valores se pueden utilizar para identificar **unívocamente** cada entidad sin lugar a duda. Por ejemplo, el atributo idCliente del conjunto de entidades cliente es suficiente para distinguir una entidad cliente de las demás.

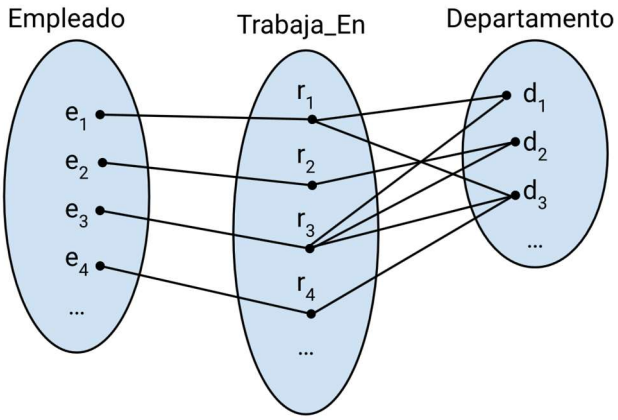
Relaciones

Una relación es una asociación entre varias entidades. Un conjunto de relaciones es un conjunto de relaciones del mismo tipo. Un tipo de relación R entre n tipos de entidades E_1, E_2, …, E_n define un conjunto de asociaciones (o un conjunto de relaciones) entre las entidades de esos tipos de entidades (posiblemente no distintas). Como en el caso de los

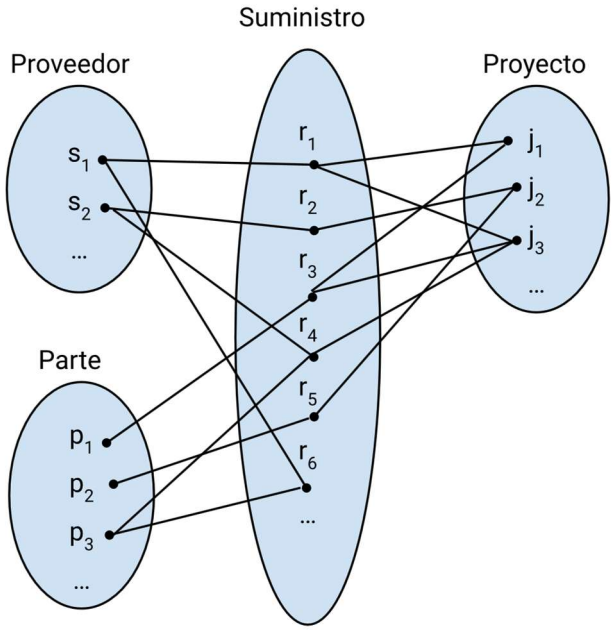
tipos de entidades y los conjuntos de entidades, normalmente se hace referencia a un **tipo de relación** y su correspondiente **conjunto de relaciones** con el mismo nombre, R.

La asociación entre conjuntos de entidades se conoce como participación; es decir, los conjuntos de entidades E_1, E_2, …, E_n **participan** en el conjunto de relaciones R.

Por ejemplo, consideremos un tipo de relación trabaja_en asociado con una entidad empleado y una entidad departamento.



El **grado de un tipo de relación es el número de tipos de entidades participantes**. Por ejemplo, la relación trabaja_para es de grado dos. Un tipo de relación de grado dos se denomina **binario**, y uno de grado tres, ternario. Un ejemplo de relación **ternaria** es suministro, donde cada relación r_i asocia tres entidades (un proveedor s, un repuesto p y un proyecto j). Las relaciones pueden ser generalmente de cualquier grado, pero las más comunes son las relaciones binarias.



La función que desempeña una entidad en una relación se denomina **rol** de esa entidad. Como los conjuntos de entidades que participan en una relación, generalmente, son distintos, los roles están implícitos y no se suelen especificar. Sin embargo, en los casos en los que instancias del mismo tipo de entidad participan en una relación con diferentes roles (**relación reflexiva o recursiva**), son necesarios **nombres explícitos** para los roles para especificar la manera en que cada una lo hace. Por ejemplo, se tiene un tipo de entidad "empleado" con la información de todos los empelados de un organismo, y un tipo de relación "trabaja_para" que relaciona dos empleados: en este caso uno tendrá el rol de trabajador, y el otro de jefe.

Una relación puede también tener atributos denominados **atributos descriptivos**. Por ejemplo, para registrar el número de horas por semana que un empleado trabaja en un proyecto en particular, podemos incluir

un atributo Horas para el tipo de relación trabaja_en. Otro ejemplo es incluir la fecha en que el director empezó a dirigir un departamento, mediante un atributo Fechalnicio para el tipo de relación administra.

Restricciones

Los tipos de **relaciones** normalmente tienen ciertas restricciones que limitan las **posibles combinaciones entre las entidades** que pueden participar en el conjunto de relaciones correspondiente. Podemos distinguir dos tipos principales de restricciones de relación: razón de **cardinalidad** y **participación**.

Cardinalidad: especifica el número **máximo** de instancias de relación en las que una entidad puede participar:

- **Uno a uno (1:1):** Cada entidad de A se asocia, a lo sumo, con una entidad de B, y cada entidad en B se asocia, a lo sumo, con una entidad de A.
- **Uno a varios (1:N):** Cada entidad de A se asocia con cualquier número (cero o más) de entidades de B. Cada entidad de B, sin embargo, se puede asociar, a lo sumo, con una entidad de A.
- **Varios a uno (N:1):** Cada entidad de A se asocia, a lo sumo, con una entidad de B. Cada entidad de B, sin embargo, se puede asociar con cualquier número (cero o más) de entidades de A.
- **Varios a varios (N:M):** Cada entidad de A se asocia con cualquier número (cero o más) de entidades de B, y cada entidad de B se asocia con cualquier número (cero o más) de entidades de A.

La correspondencia de cardinalidades adecuada para un conjunto de relaciones dado depende, obviamente, de la situación del mundo real que el conjunto de relaciones modele.

Participación: especifica si la existencia de una entidad depende de si está relacionada con otra entidad a través de un tipo de relación. Esta restricción especifica el número **mínimo** de instancias de relación en las que puede participar cada entidad, y en ocasiones recibe el nombre de restricción de **cardinalidad mínima**. Hay dos tipos de restricciones de participación, **total** y **parcial.** Se dice que la participación de un conjunto de entidades E en un conjunto de relaciones R es total si cada entidad de E participa, al menos, en una relación de R. Si sólo algunas entidades de E participan en relaciones de R, se dice que la participación del conjunto de entidades E en la relación R es parcial.

Por ejemplo, si se dice que cada empleado debe trabajar para un departamento, entonces una entidad de empleado sólo puede existir si participa en al menos una instancia de relación trabaja_para. Es decir, cada entidad de "empleado" debe estar relacionada con una entidad "departamento" a través de "trabaja_para" (participación total o **dependencia de existencia**). Por otro lado, como no esperamos que cada empleado dirija un departamento, la participación de "empleado" con la relación "administra" es parcial; sólo algunos estarán relacionados, pero no necesariamente todos.

Entidades débiles y fuertes

Los tipos de entidad que tienen **existencia por sí misma** en el ámbito de la realidad que se está modelando, independientemente de cualquier otra entidad, se denominan tipos de **entidades fuertes** (regular o común). Siempre se podrá especificar una clave para estas con sus atributos propios.

Los tipos de entidad que dependen de alguna entidad existente (**propietaria o identificatoria**), se denominan tipos de **entidades débiles**. Generalmente no tienen atributos suficientes para formar una clave primaria. Este tipo de entidades se relacionan con entidades específicas de otro tipo de entidad en combinación con uno de sus valores de atributo a través de una **relación identificativa** del tipo de entidad débil. Un tipo de entidad débil siempre tiene una restricción de participación total (**dependencia de existencia**) respecto a su relación identificativa, porque una entidad débil no puede identificarse sin una entidad propietaria. Es decir, al desaparecer una instancia de la entidad fuerte superior, desaparecerá toda instancia de la entidad débil vinculada a la misma.

Por ejemplo, consideremos el conjunto de entidades "pago" correspondientes a "préstamos", que tiene tres atributos: número_pago, fecha_pago e importe_pago. Los números de pago suelen ser números secuenciales, a partir de 1, generados independientemente para cada préstamo. Por tanto, aunque cada entidad "pago" es distinta, los pagos de diferentes préstamos pueden compartir el mismo número de pago. Así,

este conjunto de entidades no tiene clave primaria; es un conjunto de entidades débiles. Además, si una instancia de "préstamos" desaparece, los pagos vinculados dejarían de tener sentido.

Aunque los conjuntos de entidades débiles pueden no tener clave primaria, hace falta un medio para distinguir entre todas las instancias que dependen de una entidad fuerte concreta. El **discriminante** (clave parcial) es un conjunto de atributos que permite que se haga esta distinción. Por ejemplo, el discriminante del conjunto de entidades débiles *pago* es el atributo *número_pago*.

La clave primaria de un conjunto de entidades débiles se forma con la clave primaria del conjunto de entidades fuertes vinculado y el discriminante del conjunto de entidades débiles.

Características del modelo E-R extendido

Especializacion y generalización

La **especialización** es el proceso de definir un **conjunto de subclases** de un tipo de entidad, la cual recibe el nombre de **superclase** de la especialización. El conjunto de subclases que forman una especialización se define basándose en algunas características distintivas de las entidades en la superclase. Por ejemplo, las subclases {secretaria, ingeniero, técnico} son una especialización de la superclase empleado que distingue a los trabajadores en función al tipo de trabajo que desempeñan. Podemos tener varias especializaciones del mismo

tipo de entidad en función de varias características distintivas. Por ejemplo, otra especialización de empleado podría ser el conjunto de subclases {personal_fijo, personal_temporal}, las cuales distinguen a los trabajadores por el tipo de contrato que tienen.

Existen dos razones principales para incluir relaciones clase/subclase y especializaciones en un modelo de datos: Pueden existir ciertos atributos que sólo deban aplicarse a algunas entidades, pero no a toda la superclase; mientras siguen compartiendo la mayor parte de sus atributos con los otros miembros de la superclase. Por ejemplo, la subclase ingeniero puede tener el atributo específico especialización, y compartir con secretaria y técnico el resto de los **atributos heredados** de la entidad empleado. Por otro lado, algunos tipos de relaciones sólo podrían establecerse entre miembros de una subclase (por ejemplo, entre personal_temporal y empresa_tercerizada si la relación sólo es válida para este tipo de personal).

El proceso de especialización permite entonces: definir un conjunto de subclases de un tipo de entidad, establecer los atributos específicos adicionales en cada subclase, y establecer relaciones específicas adicionales entre cada subclase y otras entidades.

Usamos el término **generalización** para referirnos al proceso por el cual se define una entidad generalizada a partir de entidades individuales. Podemos pensarlo como un proceso inverso de abstracción en el que eliminemos las diferencias existentes entre distintas entidades,

identifiquemos las características comunes y las generalicemos en una única superclase de la que las entidades originales sean subclases especiales.

Existen dos tipos principales de restricciones para la generalización o especialización en el modelo entidad-relación: la **restricción de superposición** y la **restricción de completitud**. Ambas definen la relación entre una superclase (entidad general) y sus subclases (entidades especializadas).

Superposición: Esta restricción define si una entidad de la superclase puede pertenecer a más de una de las subclases al mismo tiempo. Hay dos opciones:

- **Disjunta:** Una entidad puede pertenecer a una sola subclase. Las subclases son mutuamente excluyentes. Por ejemplo, una persona puede ser un estudiante o un profesor, pero no ambos al mismo tiempo, en el contexto de un sistema específico.
- **Superpuesta:** Una entidad puede pertenecer a varias subclases simultáneamente. Por ejemplo, una persona puede ser tanto un estudiante como un empleado en la universidad si trabaja en la biblioteca mientras estudia.

Completitud: Esta restricción define si cada entidad de la superclase debe pertenecer a alguna de las subclases. Hay dos opciones:

- **Total:** Cada entidad de la superclase debe pertenecer al menos a una de las subclases. Por ejemplo, si la superclase es Vehículo, y las subclases son Auto y Camión, y todos los vehículos son necesariamente uno de esos dos tipos, la restricción es total.

- **Parcial:** Una entidad de la superclase puede no pertenecer a ninguna de las subclases. Por ejemplo, si la superclase es Persona, y las subclases son Profesor y Estudiante, la restricción es parcial, ya que puede haber personas en el sistema que no sean ni profesores ni estudiantes (por ejemplo, personal administrativo).

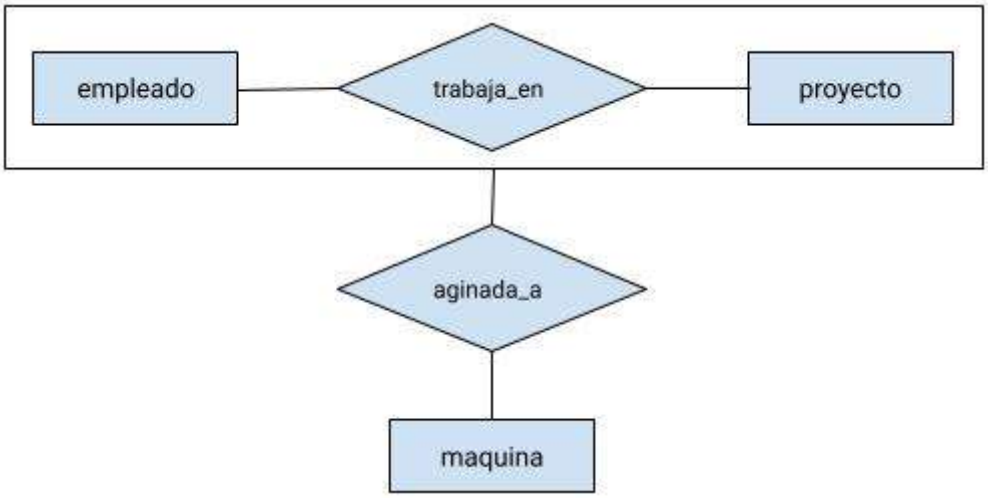
Agregación

Una limitación del modelo E-R es que no es posible expresar relaciones entre las relaciones. La agregación es una abstracción a través de la cual las **relaciones se tratan como entidades** de nivel superior.

Si tenemos una relación que asocia a las entidades A y B, la agregación permite que esta relación de A y B se agrupe en una sola "entidad agregada" (una especie de super-entidad), la cual luego puede participar en una nueva relación con una tercera entidad, C. En esencia, la agregación representa una relación **entre una entidad y otra relación**.

Por ejemplo, supongamos que tenemos la relación (N:M) “trabaja_en”, entre empleado y proyecto; y queremos modelar la asignación de cada máquina a un empleado que trabaja en un proyecto. Aquí, la asignación de la máquina no es solo al empleado o al proyecto, sino a la combinación

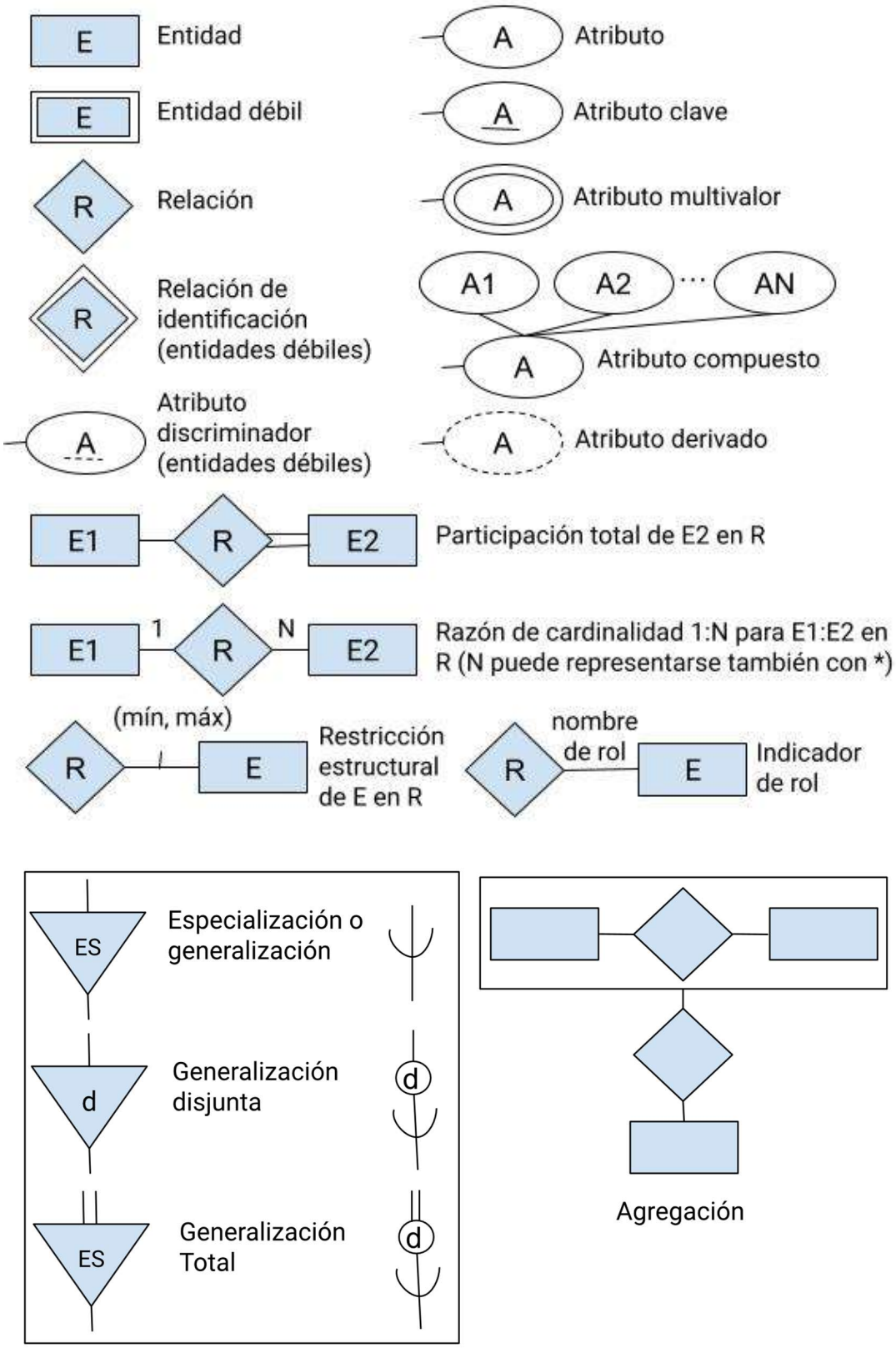
de ambos. Se puede intentar modelar esto sin agregación, con una relación ternaria (una relación entre tres entidades: Empleado, Proyecto y Máquina). Con la agregación se puede tratar la relación entre empleados y proyecto como una entidad individual (dentro de un rectángulo para su forma visual), y establecer una relación asignada_a entre esta nueva entidad y máquina.



De esta forma, el modelo deja claro que una máquina está asociada a la combinación específica de un empleado trabajando en un proyecto, y no simplemente a uno de ellos de forma independiente. La agregación ayuda a modelar esta dependencia de una manera más clara y precisa.

Diagramas Entidad-Relación (DER) como herramienta de modelado conceptual.

Los diagramas E-R pueden expresar gráficamente la estructura lógica general de las bases de datos. Los diagramas E-R son sencillos y claros, y constan de los siguientes componentes principales:


E
Entidad
A
Atributo
E
Entidad débil
A
Atributo clave
R
Relación
A
Atributo multivalor
R
Relación de identificación (entidades débiles)
A1
A2
AN
A
Atributo compuesto
A
Atributo discriminador (entidades débiles)
A
Atributo derivado
E1
R
E2
Participación total de E2 en R
E1
1
R
N
E2
Razón de cardinalidad 1:N para E1:E2 en R (N puede representarse también con *)
(mín, máx)
R
E
Restricción estructural de E en R
R
nombre de rol
E
Indicador de rol
ES
Especialización o generalización
d
Generalización disjunta
ES
Generalización Total
Agregación

El modelo de datos E-R permite gran flexibilidad en el diseño de los esquemas de las bases de datos para modelar situaciones. El diseñador de bases de datos puede escoger entre una amplia gama de alternativas. Entre las decisiones que debe tomar el diseñador están:

- Si usar atributos o conjuntos de entidades para representar cosas.
- Si los conceptos del mundo real se expresan mejor mediante conjuntos de entidades o mediante conjuntos de relaciones.
- Si usar relaciones ternarias o pares de relaciones binarias.
- Si usar conjuntos de entidades fuertes o débiles; cada conjunto de entidades fuertes y sus conjuntos de entidades débiles se puede considerar como un solo "objeto" de la base de datos, ya que las entidades débiles dependen existencialmente de la entidad fuerte.
- Si es adecuado usar la generalización; la generalización, o la jerarquía de relaciones, contribuye a la modularidad al permitir que los atributos comunes de entidades parecidas se representen en un solo sitio del diagrama E-R.
- Si es adecuado usar la agregación; la agregación agrupa parte del diagrama E-R en un solo conjunto de entidades, lo que permite tratar el conjunto de entidades agregadas como una sola unidad sin necesidad de preocuparse por los detalles de su estructura interna.

Se necesita tener una buena comprensión de la problemática que se modela para poder adoptar las diferentes decisiones de diseño necesarias.

Transformación de DER a modelo relacional.

Se puede diseñar el esquema de una base de datos relacional basándose en un diseño de esquema conceptual. Este mapeo es el proceso de convertir el diseño conceptual de una base de datos (el Diagrama E-R) en un conjunto de tablas, atributos y restricciones del modelo relacional que definen la estructura de la base de datos. Se siguen **reglas específicas** para cada componente del diagrama.

Mapeo de entidades fuertes: Por cada entidad fuerte E del esquema ER, cree una relación (tabla) R que incluya todos los atributos simples de E. Incluya únicamente los atributos simples que conforman un atributo compuesto. Seleccione los atributos clave de E como clave principal para R. Si la clave elegida de E es compuesta, entonces el conjunto de los atributos simples que la forman constituirá la clave principal de R. Los atributos multivaluados quedan excluidos en este momento.

Mapeo de entidades débiles: Por cada entidad débil W del esquema E-R con el tipo de entidad propietario E, cree una relación (tabla) R e incluya todos los atributos simples (o componentes simples de los atributos compuestos) de W como atributos de R. Además, incluya como atributos aquellos que forman la clave primaria de la entidad padre de la que depende. La clave primaria de R es la combinación de la clave primaria del propietario, y la clave parcial (discriminador) del tipo de entidad débil W, si la hubiera. Además, se indica como clave foránea (FK), la clave primaria de la entidad padre para establecer la relación.

Mapeo de relaciones: a Tablas o Claves Foráneas. La forma en que se mapea una relación depende de su cardinalidad.

- **Relaciones Uno a Uno (1:1):** No se crea una tabla nueva. La clave primaria de una de las entidades se añade como una clave foránea (foreign key) en la tabla de la otra entidad. La decisión de dónde colocar la clave foránea a menudo se basa en la participación obligatoria. Se coloca la clave foránea en la tabla de la entidad que tiene participación parcial (tiene menos posibilidades de ser nula). Además, se incluyen en esta todos los atributos que pudieran existir en la relación 1:1 como atributos de esta entidad.

Por Ejemplo: Una relación 1:1 “Dirige” entre “Empleado” y “Departamento” se mapea añadiendo la clave primaria de “Empleado” (ID_Empleado) como una clave foránea en la tabla Departamento (podría renombrarse la FK a “ID_Empleado_Director”) para darle más significado.

Otra alternativa es mezclar los dos tipos de entidad y la relación en una sola tabla en los casos en las que las dos participaciones son totales.

- **Relaciones Uno a Muchos (1:N):** La clave primaria de la entidad del lado "uno" se añade como una clave foránea en la tabla del lado "muchos". Esto es fundamental para mantener la integridad referencial. Además, se incluyen en la tabla donde se agrega la clave foránea, todos los atributos que pudieran existir en la relación 1:N.

Ejemplo: Una relación 1:N entre Departamento y Empleado (un departamento tiene muchos empleados) se mapea añadiendo la clave primaria de Departamento (ID_Departamento) como clave foránea en la tabla Empleado.

- **Relaciones Muchos a Muchos (N:M):** Se crea una nueva tabla para representar la relación. Esta se conoce como **tabla de unión** o **tabla de intersección**. Esta nueva tabla contiene como clave primaria compuesta las claves primarias de las dos entidades participantes, las cuales actúan como claves foráneas. También puede incluir atributos propios de la relación.

Ejemplo: Una relación N:M entre Estudiante y Curso (muchos estudiantes se inscriben en muchos cursos) se mapea creando una nueva tabla llamada INSCRIPCIONES con las columnas ID_Estudiante y ID_Curso. Ambas son claves foráneas y, juntas, forman la clave primaria de la tabla Inscripciones.

- **Relaciones n-ary:** Las anteriores correspondían a relaciones binarias. En este caso, por cada tipo de relación n-ary R, donde n > 2, se crea una nueva tabla S para representar R. Se incluyen como atributos de la foreign key en S las claves principales de las entidades participantes. También se agregan los atributos simples del tipo de relación n-ary (o los componentes simples de los atributos compuestos) como atributos de S. Normalmente, la clave

principal de S es una combinación de todas las foreign keys que hacen referencia a las entidades participantes.

Mapeo de Atributos Especiales: Además de los atributos simples, existen otros tipos que requieren un tratamiento específico:

- Atributos **Compuestos:** Se descomponen en sus partes atómicas. Por ejemplo, si una entidad Persona tiene un atributo compuesto Dirección con sub-atributos Calle, Ciudad y CódigoPostal, la tabla Persona tendrá tres columnas separadas: Calle, Ciudad y CódigoPostal. Esto garantiza que cada columna almacena un único valor indivisible.

- **Atributos Multivalor:** No se pueden representar como una sola columna, ya que una celda de la tabla no puede contener múltiples valores. Por lo tanto, se crea una nueva tabla para el atributo. Esta nueva tabla tendrá una columna para el atributo multivalor, más una clave foránea que referencia a la clave primaria de la entidad original que tiene dicho atributo. Por ejemplo, para un atributo Teléfono (que puede tener multivalores) en la entidad Empleado, se crea una tabla Telefonos_Empleado con las columnas ID_Empleado (clave foránea) y Telefono. La clave primaria de esta nueva tabla será la combinación de ambas columnas.

Mapeo de Generalización / Especialización: Este es uno de los mapeos más complejos, ya que existen varias estrategias. La elección depende de las restricciones de superposición y completitud.

- **Estrategia 1: Una Tabla por Jerarquía**
    - Se crea una **única tabla** para la superclase y todas sus subclases.
    - La tabla incluye todos los atributos de la superclase y de todas las subclases.
    - Se añade una columna especial (llamada discriminador) para indicar a qué subclase pertenece cada fila. **Los atributos de las subclases que no se aplican a una fila** en particular se dejan con **valores NULL**.
    - **Mejor para**: Jerarquías con **subclases que no tienen muchos atributos únicos o cuando la superclase tiene muchas subclases**.
    - **Desventajas:** Se puede desperdiciar mucho espacio debido a los **valores NULL**. Se **pierde expresividad si existen relaciones en las que no participa la entidad padre** sino un cierto subtipo.

- **Estrategia 2: Una Tabla por Superclase y Subclase**
    - Se crea **una tabla para la superclase y una tabla para cada subclase**.
    - La tabla de la superclase contiene sus atributos y su clave primaria.

- Cada tabla de subclase contiene sus atributos y la clave primaria de la superclase, que también actúa como clave primaria y clave foránea de la subclase.
- **Mejor para:** Jerarquías con **subclases que tienen muchos atributos únicos y restricciones de completitud total. Minimiza el uso de valores NULL.**
- **Desventajas:** Las consultas que **necesitan datos de la superclase y una subclase** requieren una operación de JOIN (uso de claves foráneas), lo que puede ser menos eficiente.

- **Estrategia 3: Solo Tablas para las Subclases**
  - Se crea **una tabla solo para cada subclase**.
  - Cada tabla de subclase contiene sus atributos más los atributos de la superclase.
  - No hay una tabla para la superclase en sí misma.
  - **Mejor para:** Cuando la superclase es abstracta y **las subclases son disjuntas y totales.**
  - **Desventajas:** Se produce redundancia de datos (**los atributos de la superclase se repiten en cada tabla de subclase**) y puede ser difícil de manejar.

Mapeo de Agregación: El mapeo de la agregación sigue las reglas de las relaciones, pero con un paso adicional. Si la agregación es de la relación entre la entidad A y la entidad B, que a su vez se relaciona con la entidad C, el mapeo sería:

1. Mapear la relación de las entidades A y B a una tabla (por ejemplo, A_B).
2. Mapear la relación de la tabla A_B con la entidad C. Esto se hace de la misma forma que una relación normal (1:1, 1:N o N:M), añadiendo la clave primaria de la tabla A_B como clave foránea en la tabla de C, o viceversa, según la cardinalidad.

La clave está en que la "entidad agregada" (la relación original) se trata como una tabla intermedia en el modelo relacional.